

A Personal Project: Development of Python Software for Astronomical Computer Vision

This slidepack details the development of a Python based software tool called “StarTrack”

See more here! <https://github.com/matthiasarndt/StarTrack>

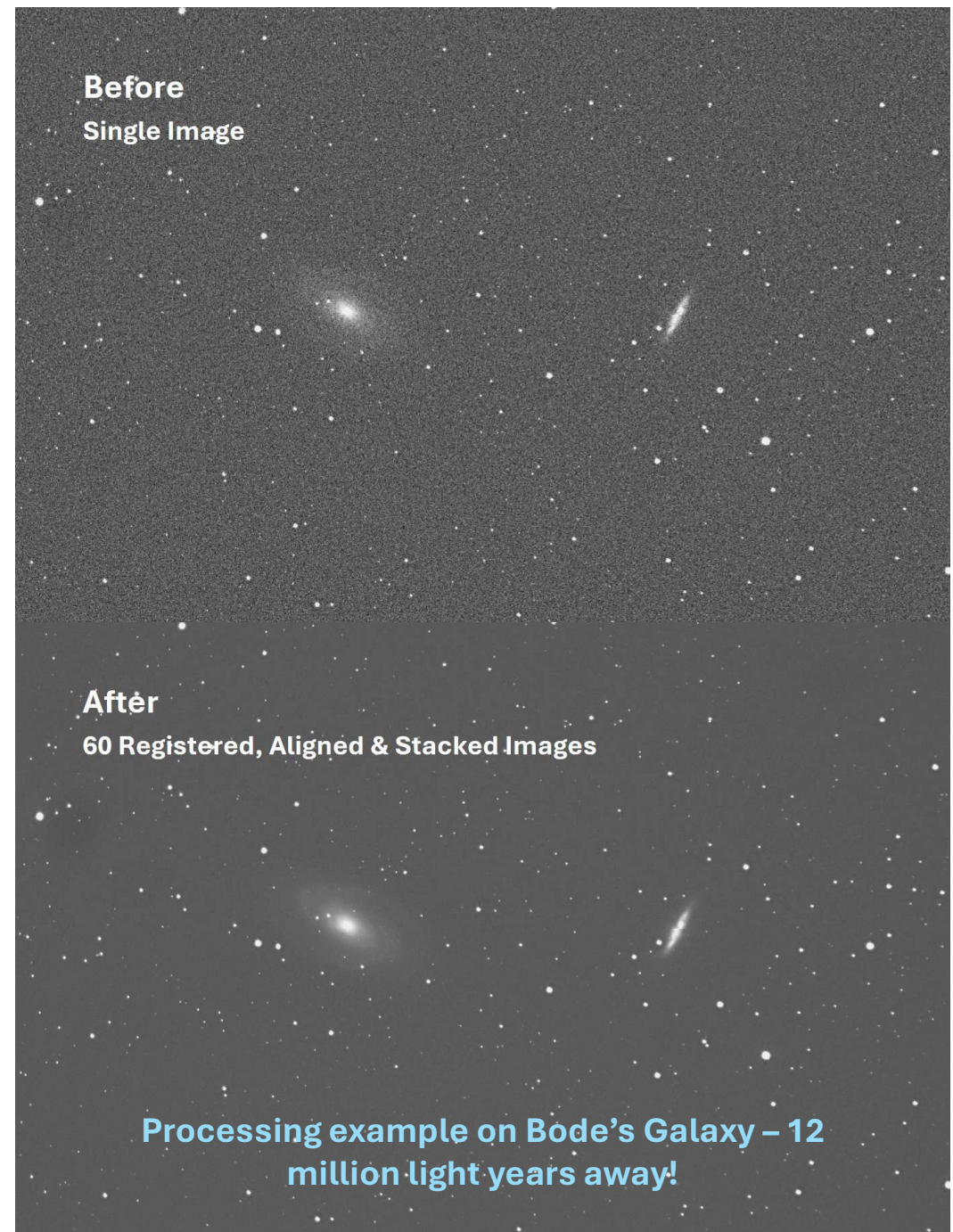
By Matthias Arndt

Contents:

- 1 : What is StarTrack?
- 2 : Which software methods have been implemented?
- 3: Why develop it?
- 4: How does StarTrack work?
- 5: How is the Astronomical data captured?
- 6: Future Roadmap
- 7: Results Showcase

StarTrack is a modular object-oriented Python package designed for astronomical Computer Vision, star detection, and noise reduction.

- See here:
<https://github.com/matthiasarndt/StarTrack>
- I have developed this code independently outside of my current engineering role, as a passion project driven by my interest in **software development, numerical modelling, and Astronomy**.
- StarTrack has been built to combine hundreds of individual images of a **Deep Space Object** (often 10s of millions of light years away), detecting and identifying stars and using them to align, layer and combine their data to reduce noise and reveal extremely faint objects!



StarTrack has been built without any Computer Vision libraries (such as OpenCV), instead relying on algorithms derived from scratch, written with [NumPy](#), [SciPy](#) and [scikit-learn](#). The following technologies have been implemented:

- **[Object-Oriented Programming](#), [Design Patterns](#) (Pipeline), Inheritance & Composition**
- **Unsupervised [Machine Learning \(sci-kit learn\)](#), to detect, register, and classify stars and Astronomical features.**
- **[Vectorised](#) numerical methods and [algorithms](#) implemented with [NumPy](#).**
- **[Optimisation](#) techniques ([SciPy](#)), used to automatically tune Image Processing parameters to allow the code to work across a large range of different image datasets.**
- **[Dataclasses](#), to ensure a traceable flow of data and immutable inputs.**
- **[Parallelisation](#) and [memory management](#) to rapidly increase processing times and reduce RAM overhead.**
- **Code optimisation to reduce overall runtime.**
- **[Regular version controlling](#) with Git onto GitHub**

Why build StarTrack? The biggest challenge in Deep Space Astronomy imaging is noise reduction.

- Many **nebulae**, **galaxies** and **star clusters** are **extremely dim**, and therefore **long exposures** are required to adequately capture their detail.
- Due to the **Earth's rotation**, it is necessary for imaging systems to **track objects in the night sky**.
- As tracking errors build up, **drift** becomes **visible** in captured images. Drift can cause stars to become distorted and for images to lose depth and clarity.

To overcome this, **hundreds of individual images are taken of a single object**, with each being exposed for a few minutes.

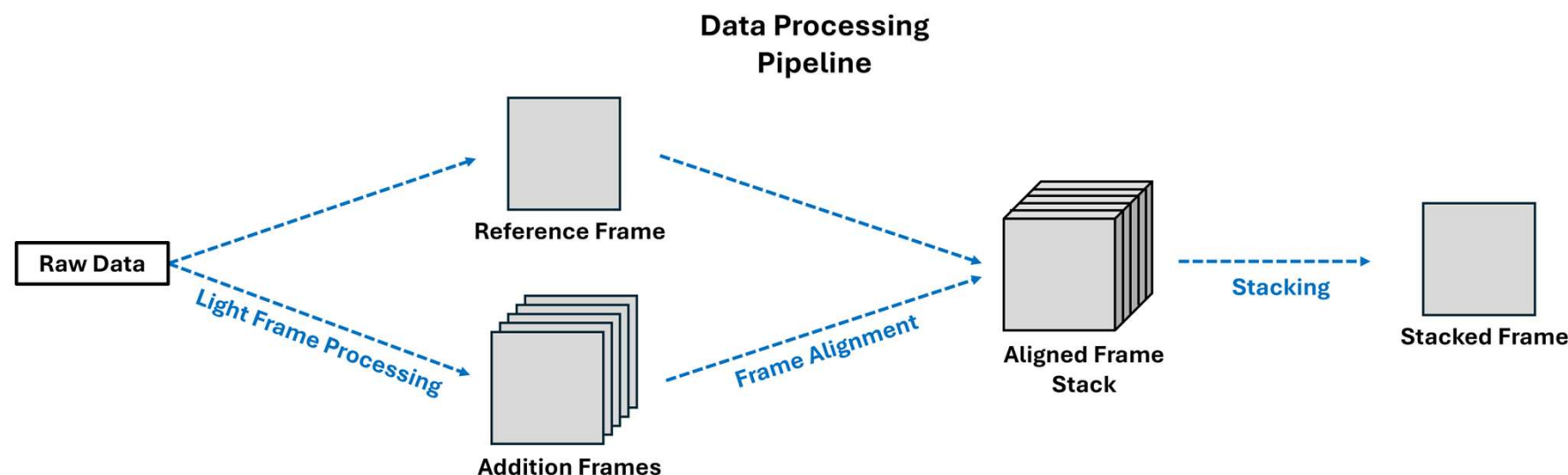
- These images are then "**stacked**" on top of each other to **reduce** the **noise** of the final stacked image – thereby providing the equivalent of one very long exposure.
- **StarTrack** is built to **combine hundreds of individual exposures** of astronomical data. It does this by running **star detection algorithms**, and identifying **reference points** across many frames, and **aligning** them.
- Using this information, it can "**stack**" these exposures together – **identifying** and **averaging** every **pixel** in each individual frame to produce a stacked exposure which has a **large reduction in noise**.

To read more about how StarTrack works, please see here:

<https://github.com/matthiasarndt/StarTrack/blob/main/README.md>

Processing is split into three key steps:

1. **Light Frame Processing:** All frames are imported and read in as LightFrame objects. These are processed to **filter**, **count**, and **catalogue** aligning **stars** inside an image.
2. **Frame Alignment:** Each of the frames (on the order of 10s-100s of images) are **aligned** with the **reference frame**, which is the initial frame that all other layers are compared against. This involves calculating reference vectors for each image and warping/translating the data to match the reference frame.
3. **Stacking:** As a result of frame alignment, a **3D array** of **data** is created, where each layer represents one input frame which has been rotated/translated to match the reference frame. **Data** at **each pixel co-ordinate** are **assessed** and **averaged** to **reduce noise** and **remove outliers**.



Astronomical Image Datasets have been **captured manually**, using an **Apochromatic Refractor** on a **motorised mount**:

- An **apochromatic refractor telescope** with a focal length of 360mm and a focal ratio of 5.8.
- A mount which can track the night sky in the Right Ascension (RA) axis.
- A Nikon DSLR which has been modified to increase sensitivity to Hydrogen Alpha wavelengths.
- See here for more:
<https://github.com/matthiasarndt/Astrophotography/blob/main/README.md>



StarTrack is currently in development, as is currently in **Version 0.2.1**. This is a general roadmap for future development & features:

1. Further **optimisation** of existing codebase.
2. Implementation of **post-processing capabilities** for final stacked images (e.g. edge detection of nebulae, and dynamic range increases to bring out features of deep space objects).
3. **Machine Learning** of final stacked image to further reduce noise using **PyTorch**. The aim here will be to use deep learning (**Variational Autoencoders – VAE**) to further reduce noise.
4. **Statistical analysis** of pixels in each layer before they are “stacked” together. This will involve understanding the distribution of each pixel and using statistical approaches to **remove noise** and increase **signal to noise ratio**.
5. **GPU based computing** to accelerate a lot of the algorithms which are currently CPU based. This will involve refactoring code to run off **CuPy** rather than **NumPy** or implementing computing with **Numba**.

A few more examples of Image Processing with StarTrack:

